

When 200 OK Isn't Success: Debugging a Broker API Integration

Problem — Every trade in a user's history displayed the same account balance, and it was the wrong number.

Root cause — The balance was fetched once, outside the trade loop, and stamped onto every trade. It also read the wrong field: cash-only, excluding open positions.

Resolution — Two viable fixes identified. Recommended reconstructing a true running balance from transaction history rather than the quick field swap, because the quick fix only solves half the reported problem.

Context

StrategyForge is a trading journal. Users connect their brokerage account through SnapTrade, an aggregator that handles the OAuth connection to the broker, and their trade history syncs automatically.

This user's broker was Robinhood. The sync worked. Trades appeared. Every API call returned 200 OK.

And every single trade showed an account balance of **\$2,561** — the same figure on all 32 trades, and not the balance of their account.

That's what made this interesting. Nothing failed. There was no error to chase, no stack trace, no red in the logs. A healthy pipeline was confidently producing wrong data.

Step 1 — Reproduce with observability

I had the customer trigger a sync while I watched the Railway deploy logs and captured the SnapTrade API responses.

Reproducing on demand matters. A bug you can only read about is a bug you're guessing at.

Step 2 — What the logs said

[SNAPTRADE] Account balance: \$2561

[SNAPTRADE] Found 32 executed orders (filtered from 35 total)

[SNAPTRADE] Grouped into 12 unique account-scoped instruments

The order counts looked healthy. The balance line was logged **once** — not once per trade. Worth noting, though I didn't appreciate how important that was yet.

Step 3 — What the API returned

GET /accounts/{id}/balances:

```
json
[
  {
    "currency": { "code": "USD", "name": "US Dollar" },
    "cash": 2561,
    "buying_power": 3875.785
  }
]
```

There it is. cash: 2561 — the exact number on every trade.

Step 4 — First hypothesis

StrategyForge is treating cash as the account balance. But a brokerage balance isn't just cash — it's cash **plus the market value of open positions**. If the user holds stock, cash understates their account by whatever those positions are worth.

That was a theory, not a finding. To confirm it I needed the value of their open positions, so I went looking through the other responses from the same sync.

Step 5 — An unexpected result

While searching for position values, I found a different response for the same account — account details rather than balances:

```
json
{
  "institution_name": "Robinhood",
  "type": "INDIVIDUAL",
  "balance": {
```

```
"total": { "amount": 4845.38, "currency": "USD" }  
  
},  
  
"sync_status": {  
  
  "transactions": {  
  
    "first_transaction_date": "2021-09-18",  
  
    "initial_sync_completed": true  
  
  }  
  
},  
  
"status": "open"  
  
}
```

4845.38. That's the correct balance — the one the user expects to see.

Step 6 — Confirming the hypothesis with arithmetic

Rather than assume the two numbers were related, I checked:

```
4845.38  total account value  
- 2561.00  cash  
_____  
2284.38  ← unaccounted for
```

That difference is the market value of the open positions — exactly what my step 4 hypothesis predicted was missing. The theory held.

I also verified both figures belonged to the **same account**, not two accounts under the same login. The user does have multiple Robinhood accounts, and it would have been easy to spend an hour on multi-account handling. That wasn't the cause: it's one account exposed through two endpoints measuring two different things.

Step 7 — Reading the API contract

Confirmed against SnapTrade's documentation:

- `/balances` returns cash and buying_power. It has **no** total field.
- Total account value lives on the account details response as `balance.total.amount`.

This mattered, because of what I found next in the code.

Step 8 — Reading the source

Two lines, roughly 400 apart in the sync routine:

```
js
```

```
// Fetched ONCE per account, before any trades are constructed
```

```
accountBalance = balances[0].cash || balances[0].total || null;
```

```
// ...~400 lines later, inside the loop that builds each trade...
```

```
balance: accountBalance ? String(accountBalance) : null,
```

Three separate defects:

- 1. Wrong field.** cash excludes position value — confirmed by the arithmetic above.
- 2. Dead fallback.** `|| balances[0].total` can never fire. That endpoint has no total field, so the fallback silently does nothing. It reads like a safety net and isn't one.
- 3. Wrong data source entirely.** The fetch sits *outside* the trade loop. One value is written to all 32 trades. That's why every row is identical.

The third one is the real bug.

Step 9 — Why the obvious fix isn't enough

The tempting fix is to swap cash for balance.total.amount. It's a one-line change and it produces the number the user expects.

It also only solves half the complaint. The original report was:

"This is not the balance of the account **and it is the same balance on every single trade.**"

Swap the field and all 32 trades change from \$2,561 to \$4,845.38 — still identical on every row. The user comes straight back.

The column is defined in the schema as *"account balance at time of trade"* and the UI labels it *"account balance after this trade."* Those are **historical, per-trade** values. Any current-balance endpoint returns a **single present-day snapshot**. No choice of field fixes that, because the information isn't in the response.

That reframed the problem: this wasn't a wrong-field bug. It was a wrong-data-source bug wearing a wrong-field bug as a disguise.

Step 10 — Two solutions

Option A — use the correct field and relabel the column. Switch to `balance.total.amount`, rename the column to "Current Account Value." One accurate number, honestly labelled. Ships immediately. Abandons the per-trade feature.

Option B — compute a real running balance. Keep "balance after trade" and calculate it properly by replaying the account's transaction history chronologically. Considerably more work, and it depends on the transactions endpoint providing sufficient history — this user's data shows `first_transaction_date: 2021-09-18`, which suggests it does.

Recommendation: Option B. Option A makes the number correct while leaving the column meaningless — arguably worse, because a plausible wrong answer invites more trust than an obviously wrong one. A trading journal that can't show how an account evolved over time is missing the point of a trading journal.

What I'd take from this

A 200 is not a correctness check. Every request in this pipeline succeeded. Status codes tell you the transport worked, not that the answer is right.

Read the field definitions, not just the field names. `cash` and `balance` sound interchangeable. They differ by the entire value of the user's holdings.

Check where a value is fetched, not just what it contains. The identical-on-every-row symptom pointed at loop placement long before it pointed at field choice. The single [SNAPTRADE] Account balance log line was the clue, and I walked past it.

Verify hypotheses numerically when you can. Subtracting two numbers turned "I think cash excludes positions" into a confirmed finding in about ten seconds.

Let evidence kill your theories. I started at "wrong field," found the right value in a different response, and ended up concluding the whole approach was wrong. The investigation was worth more than the first fix would have been.

Beware fallbacks that can't fire. `|| balances[0].total` looks like defensive coding. It's dead code that makes a fragile line look robust.

Account figures are from a real user sync, shared with permission. Identifiers removed.